Entacher, K. 1997, "A Collection of Selected Pseudorandom Number Generators with Linear Structures", Technical Report No. 97, Austrian Center for Parallel Computation, University of Vienna. [Available on the Web at multiple sites.][2]

Knuth, D.E. 1997, *Seminumerical Algorithms*, 3rd ed., vol. 2 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley).[3]

Schrage, L. 1979, "A More Portable Fortran Random Number Generator," *ACM Transactions on Mathematical Software*, vol. 5, pp. 132–138.[4]

Park, S.K., and Miller, K.W. 1988, "Random Number Generators: Good Ones Are Hard to Find," *Communications of the ACM*, vol. 31, pp. 1192–1201.[5]

L'Ecuyer, P. 1997 "Uniform Random Number Generators: A Review," *Proceedings of the 1997 Winter Simulation Conference*, Andradóttir, S. et al., eds. (Piscataway, NJ: IEEE).[6]

Marsaglia, G. 1999, "Random Numbers for C: End, at Last?", posted 1999 January 20 to sci.stat.math.[7]

Marsaglia, G. 2003, "Diehard Battery of Tests of Randomness v0.2 beta," 2007+ at `http://www.cs.hku.hk/~diehard/`.[8]

Rukhin, A. et al. 2001, "A Statistical Test Suite for Random and Pseudorandom Number Generators", NIST Special Publication 800-22 (revised to May 15, 2001).[9]

Marsaglia, G. 2003, "Xorshift RNGs", *Journal of Statistical Software*, vol. 8, no. 14, pp. 1-6.[10]

Brent, R.P. 2004, "Note on Marsaglia's Xorshift Random Number Generators", *Journal of Statistical Software*, vol. 11, no. 5, pp. 1-5.[11]

L'Ecuyer, P. 1996, "Maximally Equidistributed Combined Tausworthe Generators," *Mathematics of Computation*, vol. 65, pp. 203-213.[12]

L'Ecuyer, P. 1999, "Tables of Maximally Equidistributed Combined LSFR Generators," *Mathematics of Computation*, vol. 68, pp. 261-269.[13]

Couture, R. and L'Ecuyer, P. 1997, "Distribution Properties of Multiply-with-Carry Random Number Generators," *Mathematics of Computation*, vol. 66, pp. 591-607.[14]

L'Ecuyer, P. 1999, "Tables of Linear Congruential Generators of Different Sizes and Good Lattice Structure", *Mathematics of Computation*, vol. 68, pp. 249-260.[15]

7.2 Completely Hashing a Large Array

We introduced the idea of a random hash or *hash function* in §7.1.4. Once in a while we might want a hash function that operates not on a single word, but on an entire array of length M. Being perfectionists, we want every single bit in the hashed output array to depend on every single bit in the given input array. One way to achieve this is to borrow structural concepts from algorithms as unrelated as the Data Encryption Standard (DES) and the Fast Fourier Transform (FFT)!

DES, like its progenitor cryptographic system LUCIFER, is a so-called "block product cipher" [1]. It acts on 64 bits of input by iteratively applying (16 times, in fact) a kind of highly nonlinear bit-mixing function. Figure 7.2.1 shows the flow of information in DES during this mixing. The function g, which takes 32 bits into 32 bits, is called the "cipher function." Meyer and Matyas [1] discuss the importance of the cipher function being nonlinear, as well as other design criteria.

DES constructs its cipher function g from an intricate set of bit permutations and table lookups acting on short sequences of consecutive bits. For our purposes, a different function g that can be rapidly computed in a high-level computer language is preferable. Such a function probably weakens the algorithm cryptographically. Our purposes are not, however, cryptographic: We want to find the fastest g, and the smallest number of iterations of the mixing procedure in Figure 7.2.1, such that our output random sequence passes the tests that are customarily applied to random number generators. The resulting algorithm is not DES, but rather a kind of "pseudo-DES," better suited to the purpose at hand.

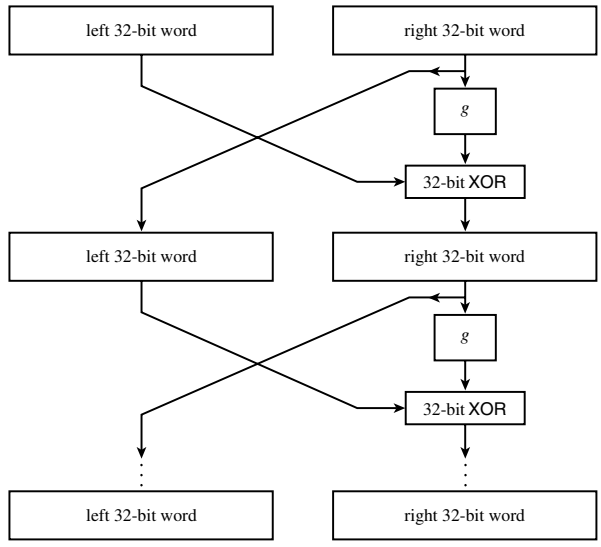

Figure 7.2.1. The Data Encryption Standard (DES) iterates a nonlinear function g on two 32-bit words, in the manner shown here (after Meyer and Matyas [1]).

Following the criterion mentioned above, that g should be nonlinear, we must give the integer multiply operation a prominent place in g. Confining ourselves to multiplying 16-bit operands into a 32-bit result, the general idea of g is to calculate the three distinct 32-bit products of the high and low 16-bit input half-words, and then to combine these, and perhaps additional fixed constants, by fast operations (e.g., add or exclusive-or) into a single 32-bit result.

There are only a limited number of ways of effecting this general scheme, allowing systematic exploration of the alternatives. Experimentation and tests of the randomness of the output lead to the sequence of operations shown in Figure 7.2.2. The few new elements in the figure need explanation: The values C_1 and C_2 are fixed constants, chosen randomly with the constraint that they have exactly 16 1-bits and 16 0-bits; combining these constants via exclusive-or ensures that the overall g has no bias toward 0- or 1-bits. The "reverse half-words" operation in Figure 7.2.2 turns out to be essential; otherwise, the very lowest and very highest bits are not properly mixed by the three multiplications.

It remains to specify the smallest number of iterations N_{it} that we can get away with. For purposes of this section, we recommend $N_{it} = 2$. We have not found any statistical deviation from randomness in sequences of up to 10^9 random deviates derived from this scheme. However, we include C_1 and C_2 constants for $N_{it} \leq 4$.

```
void psdes(Uint &lword, Uint &rword) {                                  hashall.h
```
Pseudo-DES hashing of the 64-bit word `(lword,rword)`. Both 32-bit arguments are returned hashed on all bits.
```
    const int NITER=2;
    static const Uint c1[4]={
        0xbaa96887L, 0x1e17d32cL, 0x03bcdc3cL, 0x0f33d1b2L};
    static const Uint c2[4]={
        0x4b0f3b58L, 0xe874f0c3L, 0x6955c5a6L, 0x55a7ca46L};
    Uint i,ia,ib,iswap,itmph=0,itmpl=0;
    for (i=0;i<NITER;i++) {
```
Perform `niter` iterations of DES logic, using a simpler (noncryptographic) nonlinear function instead of DES's.
```
        ia = (iswap=rword) ^ c1[i];
        itmpl = ia & 0xffff;
        itmph = ia >> 16;
```
The bit-rich constants `c1` and (below) `c2` guarantee lots of nonlinear mixing.

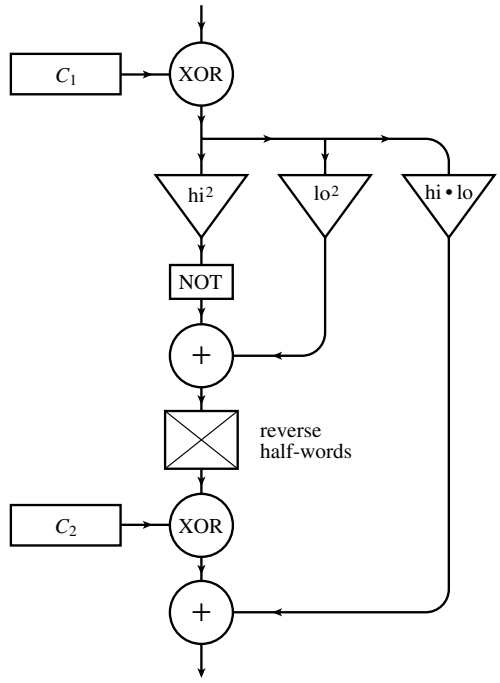

Figure 7.2.2. The nonlinear function g used by the routine `psdes`.

```
        ib=itmpl*itmpl+ ~(itmph*itmph);
        rword = lword ^ (((ia = (ib >> 16) |
            ((ib & 0xffff) << 16)) ^ c2[i])+itmpl*itmph);
        lword = iswap;
    }
}
```

Thus far, this doesn't seem to have much to do with completely hashing a large array. However, `psdes` gives us a building block, a routine for mutually hashing two arbitrary 32-bit integers. We now turn to the FFT concept of the *butterfly* to extend the hash to a whole array.

The butterfly is a particular algorithmic construct that applies to an array of length N, a power of 2. It brings every element into mutual communication with every other element in about $N \log_2 N$ operations. A useful metaphor is to imagine that one array element has a disease that infects any other element with which it has contact. Then the butterfly has two properties of interest here: (i) After its $\log_2 N$ stages, everyone has the disease. Furthermore, (ii) after j stages, 2^j elements are infected; there is never an "eye of the needle" or "necking down" of the communication path.

The butterfly is very simple to describe: In the first stage, every element in the first half of the array mutually communicates with its corresponding element in the second half of the array. Now recursively do this same thing to each of the halves, and so on. We can see by induction that every element now has a communication path to every other one: Obviously it works when $N = 2$. And if it works for N, it must also work for $2N$, because the first step gives every element a communication path into both its own and the other half of the array, after which it has, by assumption, a path everywhere.

We need to modify the butterfly slightly, so that our array size M does not have to be a power of 2. Let N be the next larger power of 2. We do the butterfly on the (virtual) size N, ignoring any communication with nonexistent elements larger than M. This, by itself, doesn't do the job, because the later elements in the first $N/2$ were not able to "infect" the second $N/2$ (and similarly at later recursive levels). However, if we do one extra communication between